

京东前端面试-AI Coding方向

一面（技术基础） · 二面（深入原理与项目） 三面（架构与综合能力）

一面（技术基础） · 1-6

1. 请介绍一下 CSS 中的 Flex 布局，以及如何实现一个典型的 AI 聊天对话框布局？
2. 在处理 AI 流式输出时，SSE 和 WebSocket 有什么区别？你通常怎么选择？
3. 请解释一下 JavaScript 的事件循环（Event Loop）机制，特别是它如何影响 UI 渲染？
4. 谈谈你对 Promise 和 async/await 的理解，如何处理多个 AI 接口的并发请求？
5. 如何在前端实现一个简单的 Markdown 渲染，并支持代码块高亮？
6. 手写题：请实现一个带有“立即执行”选项的防抖函数（Debounce）。

二面（深入原理与项目） 1-8

1. AI 聊天界面中，当消息非常多时会出现卡顿，你会从哪些维度进行性能优化？
2. 请详细说明如何实现 AI 输出的“打字机效果”，并解决流式数据解析不完整的问题。
3. 在开发 AI Coding 工具时，如何设计 Prompt 模板以保证模型返回稳定的 JSON 格式？
4. 谈谈 React 的 Fiber 架构，它对处理 AI 这种高频更新的场景有什么优势？
5. 前端如何实现一个“代码对比预览”组件（类似 Git Diff）？
6. 你们在项目中是如何处理 AI 接口的超时和重试机制的？
7. 手写题：实现一个带并发限制的异步调度器 Scheduler。
8. 如果要实现一个前端本地的“向量检索”功能，你会怎么做？

三面（架构与综合能力） 1-5

1. 让你设计一个 RAG（检索增强生成）的前端全链路方案，你会如何架构？
2. 在 AI 时代，你认为前端组件库的设计会发生哪些变化？
3. 如何衡量一个 AI 辅助编程工具（如京东内部的 AI 助手）的业务价值？
4. 谈谈你在项目中遇到的最难的 AI 相关技术挑战，你是如何解决的？
5. 你如何看待 WebAssembly（Wasm）在未来前端 AI 领域的地位？

一面（技术基础）

1. SSE 和 WebSocket 有什么区别？如何选择？

- **SSE (Server-Sent Events)**: 基于 HTTP, **单向通信** (服务端 -> 客户端)。天然支持断线重连, 协议轻量, 由于是文本协议, 更适合处理 AI 的流式文本输出。
- **WebSocket**: **全双工通信** (双向)。协议相对复杂, 适用于需要高频双向交互 (如实时协作、语音通话) 的场景。
- **选择依据**: 若只需展示 AI 生成过程, **SSE 是首选** (兼容性好、开发成本低); 若涉及实时双向对弈、复杂的游戏化交互, 选 WebSocket。

3. JavaScript 事件循环机制及其对 UI 渲染的影响?

- **核心机制**: JS 是单线程的, 分为 **宏任务 (Macrotask)** 和 **微任务 (Microtask)**。每一轮宏任务执行完后, 会清空当前所有的微任务。
- **UI 渲染时机**: 浏览器通常在 **微任务队列清空后、下一轮宏任务开始前** 尝试进行渲染。
- **影响**: 如果微任务 (如大量的 `Promise.then`) 过多或宏任务执行时间过长, 会阻塞渲染线程, 导致界面卡顿。在 AI 流式输出中, 如果频繁触发状态更新, 需注意控制更新频率, 避免阻塞渲染。

4. Promise 和 async/await 理解及并发请求处理?

- **理解**: `Promise` 是异步编程的解决方案; `async/await` 是其语法糖, 使异步代码看起来像同步代码, 提高可读性。
- **并发处理**:
 - `Promise.all`: 所有接口都成功才返回, 适合强依赖场景。
 - `Promise.allSettled`: 无论成功失败都返回结果, 适合多个 AI 插件并行调用。
 - **控制并发**: 若需限制并发数 (如一次只发 3 个请求), 需要实现一个带 `limit` 的调度器。

5. 如何实现简单的 Markdown 渲染及代码高亮?

- **Markdown 解析**: 通常使用 `marked` 或 `markdown-it` 库。
- **代码高亮**: 结合 `Prism.js` 或 `highlight.js`。
- **流程**:
 - a. 将 Markdown 字符串传入解析器。
 - b. 使用解析器的 `highlight` 配置项, 匹配代码块标签。
 - c. 引入对应的 CSS 主题文件。

6. 手写题: 带“立即执行”选项的防抖函数

代码块

```
1 function debounce(fn, wait, immediate = false) {
```

```
2   let timer = null;
3   return function(...args) {
4     if (timer) clearTimeout(timer);
5     if (immediate && !timer) {
6       fn.apply(this, args);
7     }
8     timer = setTimeout(() => {
9       if (!immediate) fn.apply(this, args);
10      timer = null; // 重置
11    }, wait);
12  };
13 }
```

二面（深入原理与项目）

1. AI 聊天界面消息过多导致卡顿，如何优化？

- **虚拟列表 (Virtual List)**: 只渲染可视区域的消息，减少 DOM 节点数量（核心方案）。
- **分片渲染**: 初始化历史记录时，利用 `requestIdleCallback` 分批插入。
- **状态平铺**: 避免深度嵌套的 state 结构，减少 React/Vue 的 Diff 耗时。
- **资源懒加载**: 图片、Markdown 插件、公式解析库按需加载。

2. 如何实现“打字机效果”及流式数据不完整处理？

- **实现**: 通过 SSE 接收 `chunk`，累加到字符串变量，界面根据此变量渲染。
- **数据不完整解析**: AI 返回的 chunk 可能是半个 JSON。
 - 使用 `TextDecoder` 处理二进制流。
 - 使用 **正则匹配** 或 **JSON 状态机 (partial-json-parser)** 尝试解析，若解析失败则缓存该 chunk，等待下一个 chunk 拼接后重试。

3. 如何设计 Prompt 以保证稳定的 JSON 格式？

- **明确约束**: 在 System Prompt 中加入 `Response in valid JSON format only`。
- **Few-shot**: 给出 1-2 个具体的输入输出 JSON 示例。
- **Schema 定义**: 提供 JSON Schema 描述字段含义。
- **防御编程**: 前端对返回内容进行正则截取（提取 `{...}` 之间的内容）并配合 `try-catch`。

4. React Fiber 对处理 AI 高频更新的优势？

- **时间切片 (Time Slicing)**: AI 流式输出频率极高, Fiber 可以将更新任务拆分, 在浏览器空闲时执行, 防止阻塞用户输入 (如在 AI 回复时用户点击停止按钮)。
- **优先级调度**: 用户交互 (点击、输入) 的优先级高于 AI 文本渲染, 保证响应灵敏。

5. 前端实现 “代码对比预览” 组件

- **原理**: 计算两个字符串的 Diff 差异 (使用 `diff` 库), 获取每一行的新增/删除/修改状态。
- **渲染**: 左侧显示 Old, 右侧显示 New, 通过 CSS 背景色 (红/绿) 区分。
- **方案**: 成熟项目可直接使用 `react-diff-viewer`。

6. 处理 AI 接口的超时与重试机制

- **超时**: 使用 `AbortController` 设置超时信号, 超过 30s 自动中断请求。
- **重试**: 配合 **指数退避算法 (Exponential Backoff)**, 在网络波动时尝试 2-3 次, 并区分错误码 (如 429 频率限制不立即重试)。

7. 手写题: 带并发限制的异步调度器

代码块

```
1  class Scheduler {
2    constructor(limit) {
3      this.limit = limit;
4      this.count = 0;
5      this.queue = [];
6    }
7    add(fn) {
8      return new Promise(resolve => {
9        this.queue.push({ fn, resolve });
10       this.run();
11     });
12   }
13   run() {
14     while (this.count < this.limit && this.queue.length) {
15       const { fn, resolve } = this.queue.shift();
16       this.count++;
17       fn().then(() => {
18         this.count--;
19         resolve();
20         this.run();
21       });
22     }
23   }
24 }
```

8. 前端本地实现“向量检索”

- **Embeddings**: 使用 Transformers.js 在浏览器端将文本转化为向量。
- **计算**: 计算向量间的 **余弦相似度**。
- **库**: 引入 `Voy` 或 `Orama` 等轻量级 WASM 向量数据库进行搜索。

三面（架构与综合能力）

1. 设计一个 RAG 前端全链路方案

- **上游**: 文件上传与解析 (PDF/Markdown) 。
- **核心逻辑**:
 - a. 前端触发向量化存储。
 - b. 搜索阶段: 输入 -> 语义搜索 -> 获取 Context -> 拼接 Prompt -> LLM。
- **前端展现**: 引用溯源 (点击 AI 回复中的脚注, 高亮显示原始文档位置) 。

2. AI 时代前端组件库的变化

- **智能化**: 组件自带 AI 属性 (如 `Input` 组件支持自动补全、`Table` 支持自然语言筛选) 。
- **原子化增强**: 组件需要更容易被 LLM 理解 (语义化的 Props 和良好的文档) 。
- **交互形态**: 从“菜单式交互”向“对话式交互”转型, 增加更多 Headless UI。

3. 如何衡量 AI 辅助工具的业务价值?

- **效率提升**: 需求交付周期缩短、单位代码产出率。
- **质量指标**: 代码采纳率 (Acceptance Rate)、Bug 修复率。
- **用户体验**: 响应延迟、用户留存、用户反馈的满意度分值。

4. 最难的技术挑战（示例）

- **挑战**: 超长流式文本导致的页面内存溢出和渲染卡顿。
- **解决**: 引入虚拟滚动 + 自研 Markdown 增量解析逻辑, 通过 Web Worker 处理复杂的正则匹配, 避免主线程阻塞。

5. WebAssembly (Wasm) 在前端 AI 领域的地位?

- **地位**: 它是前端 AI 的 **性能基石**。
- **作用**: LLM 模型量化后可以在浏览器运行, Wasm 加速了矩阵运算 (相比纯 JS 提升 10 倍以上) 。

- **未来：**使得“隐私敏感”和“离线可用”的 AI 应用成为可能，减少昂贵的 GPU 资源开销。